



SPRING DATA



Guía Paso a Paso – Crear un DAO con Spring Data JPA y recuperar personas

Introducción

En esta lección lograrás:

- ✓ Crear una clase DAO usando una `interface`
- ✓ Inyectar el DAO al controlador usando `@Autowired`
- ✓ Usar `findAll()` para recuperar registros
- ✓ Ver los datos en la vista con Thymeleaf

Arquitectura a utilizar:



🧬 Paso 1: Crear la interfaz DAO

📁 Ubicación:

Crea una nueva interfaz en el paquete:

```
src/main/java/gm/dao/PersonaDao.java
```

🏠 Código de la interfaz:

```
package gm.dao;

import gm.domain.Persona;
import org.springframework.data.repository.CrudRepository;

public interface PersonaDao extends CrudRepository<Persona, Long>{

}
```

 Detalles:

- `CrudRepository<Persona, Long>`: esta interfaz ya incluye métodos como `findAll()`, `save()`, `deleteById()` y más.
 - **No necesitas una clase de implementación.** Spring Boot la genera automáticamente en tiempo de ejecución.
-

 Paso 2: Inyectar la interfaz DAO en el controlador

Abre:

```
src/main/java/gm/web/InicioControlador.java
```

 Agrega la inyección de dependencias:

```
@Autowired
private PersonaDao personaDao;
```

 Recupera la lista de personas y compártela con la vista:

```
@GetMapping("/")
public String inicio(Model model) {
    log.info("Ejecutando el controlador Spring MVC");
    var personas = personaDao.findAll();
    model.addAttribute("personas", personas);
    return "index";
}
```

 Código Final – InicioControlador.java

```
package gm.web;

import gm.dao.PersonaDao;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class ControladorInicio {
```

```

private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

@Autowired
private PersonaDao personaDao;

@GetMapping("/")
public String inicio(Model model) {
    log.info("Ejecutando el controlador Spring MVC");
    var personas = personaDao.findAll();
    model.addAttribute("personas", personas);
    return "index";
}
}

```

Paso 3: Ajustar la vista para mostrar los datos

Abre:

src/main/resources/templates/index.html

▼ Asegúrate de que solo esté el código para mostrar la tabla de personas. Puedes dejarlo así:

```

<h2>Listado de Personas</h2>
<div th:if="{personas != null and !personas.empty}">
    <table border="1">
        <tr>
            <th>Nombre</th>
            <th>Apellido</th>
            <th>Email</th>
            <th>Telefono</th>
        </tr>
        <tr th:each="persona : {personas}">
            <td th:text="{persona.nombre}">Nombre</td>
            <td th:text="{persona.apellido}">Apellido</td>
            <td th:text="{persona.email}">Email</td>
            <td th:text="{persona.telefono}">Telefono</td>
        </tr>
    </table>
</div>
<div th:if="{personas == null or personas.empty}">
    La lista de personas está vacía
</div>

```

Paso 4: Ejecutar y probar

 Ejecuta la clase `HolaSpringApplication.java` ubicada en:

```
src/main/java/gm/HolaSpringApplication.java
```

 Luego ve a tu navegador y abre:

<http://localhost:8080>

 Deberías ver una tabla con las personas registradas en tu base de datos.

 Prueba agregar un nuevo registro directamente en tu base de datos (por ejemplo, con MySQL Workbench), y luego refresca el navegador. Verás el nuevo dato automáticamente gracias al uso de `findAll()`.

Conclusión

Con esta lección:

- ✓ Creaste un DAO basado en `CrudRepository`
- ✓ Recuperaste datos sin escribir una sola consulta SQL
- ✓ Mostraste esos datos en la vista usando Thymeleaf

 SOON En la siguiente lección, agregarás la **capa de servicio** para aplicar el concepto de transacciones y lógica de negocio.

Sigue adelante con tu aprendizaje  , ¡el esfuerzo vale la pena!

¡Saludos! 

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)